Future Computing Technologies Lab: Creative Inquiry

SEMESTER REPORT

December 10, 2025

Zachary Elias
Clemson University
Department of Electrical and Computer Engineering
zkelias@clemson.edu

1 Introduction

For my project, I worked with satellite images to create a model that was able to detect roads. I used two datasets, both of which are from previous online challenges. I used the DeepGlobe Road Extraction Dataset, which has high resolution images of the Middle East, and the Payne dataset, which contains satellite photos of U.S. neighborhoods from the AIcrowd Road Segmentation Dataset. These datasets gave me two completely different environments to work with, allowing me to test how well the model could adapt to different settings. The technique I used in this project is called systematic segmentation, in which the model classifies every pixel of an image. The goal was to create a bianary mask over the image, showing where the roads are (1 = road, 0 = not road). To do this, I trained a U-Net model with two different training strategies: one based on accuracy and the other on Intersection over Union (IoU). I chose this project because, although it was fairly simple, it allowed me to become familiar with computer vision, road segmentation, and the U-Net model.

2 Materials and Methods

The first step was organizing both the DeepGlobe and Payne datasets into a format that was best for training. I created file paths for each image and mask pair and stored them in CSV files. Using these CSV files, I built a Tensorflow input pipeline that did data loading, preprocessing, batching, and shuffling. I loaded every image and mask from the disk, decoded them, and resized them to 256x256 pixels to make sure the datasets were consistent. I also changed the satellite images to fit in a range between 0-1 and the masks were analyzed so that the road pixels became 1 and the background pixels became 0. After processioning, I created a `tf.data.Dataset` object that combined the images and masks. This made it easier for Tensorflow to handle actions like loading and shuffling the data in the background so the model doesn't have to wait for data.

The model architecture that I chose for this project was U-Net, which is a convolutional neural network specifically designed for semantic segmentation. The U-Net encoder (left side of Figure 1) is made of convolutional networks and pooling, which shrink the image every layer and help the model learn different features. The decoder (right side of Figure 1) rebuilds the images using transposed convolutions and outputs a predicted mask. Skip connections (arrows in Figure 1) link each encoder block to its matching decoder block and transfer important details like thin roads and curves that would get lost by shrinking the image. In my U-Net, the encoder and decoder used 64, 128, 256, and 512 filters, with a 1024-filter bottleneck (bottom of the "U") in the middle.

To train the model, I had to choose a way to measure loss and metrics. I trained two different versions of the model to compare different optimization goals. One version I used was called binary cross-entropy (BCE) and tracked accuracy, which measures all the pixels in the image and outputs a percent of which ones are measured correctly. The second verson used a combined IoU + Dice loss, which is better for segmentation because it only measures the exact overlap between the predicted and true shapes. Both models

used the Adam optimizer with default parameters. I trained both models with a batch size of 4 and trained the accuracy model for 20 epochs. Unfortunately, I ran into GPU issues with Palmetto's service and was forced to work on CPU for most of the project. Because of this I was only able to complete 5 epochs of the IoU model.

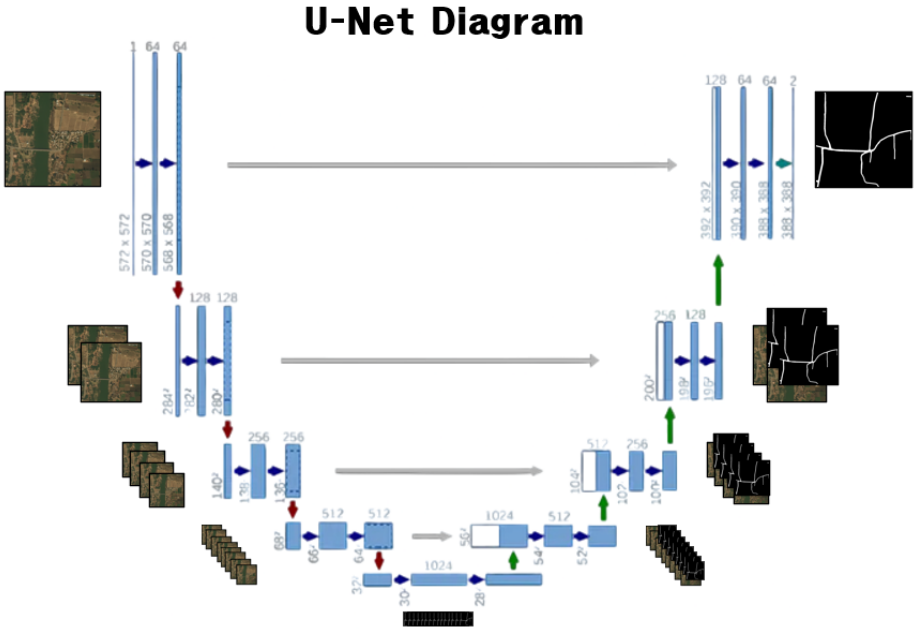

Figure 1: U-Net Diagram of Encoder, Decoder, and Skip Connections

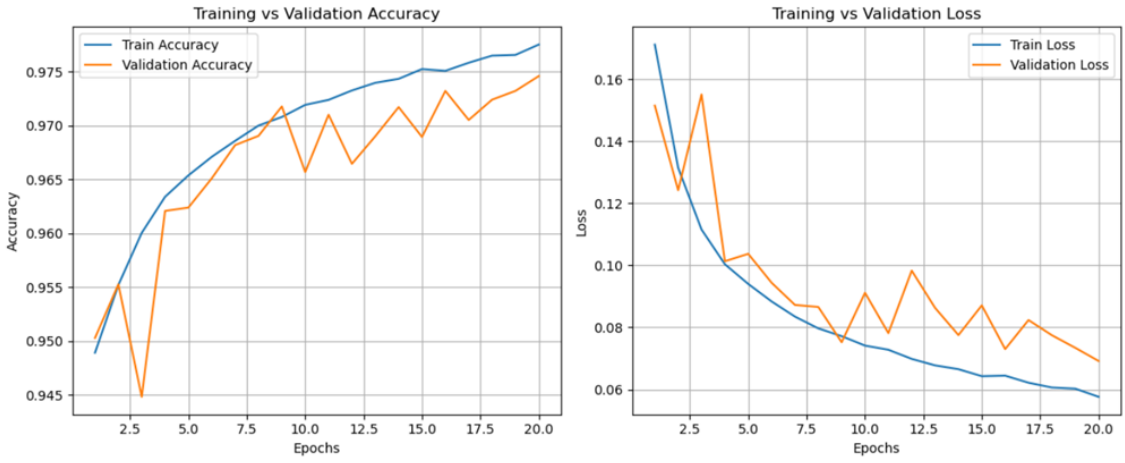

Figure 2: Training vs. Validation Accuracy and Loss for the BCE Model

3 Results

After evaluating both of the accuracy and IoU trained programs, the results showed that the models behaved differently, even when their accuracy appears nearly identical.

Table 1: Accuracy vs IoU Results

Category	**Accuracy Model**	**IoU Model**
Accuracy Score	0.9747	0.9749
IoU Score	0.4683	0.5246

Looking at the table, both models almost have the same accuracy score. The accuracy-trained model reached an IoU of 0.4693, while the IoU-trained version improved to 0.5246 and would have improved more if I completed more epochs. Since IoU measures how much the predicted mask overlaps with the ground truth, this shows that the model captured the overall shape of the roads better. The higher IoU also shows that the model was able to identify smaller and thinner roads more easily than the accuracy model. Figure 3 compares both models on two test samples:

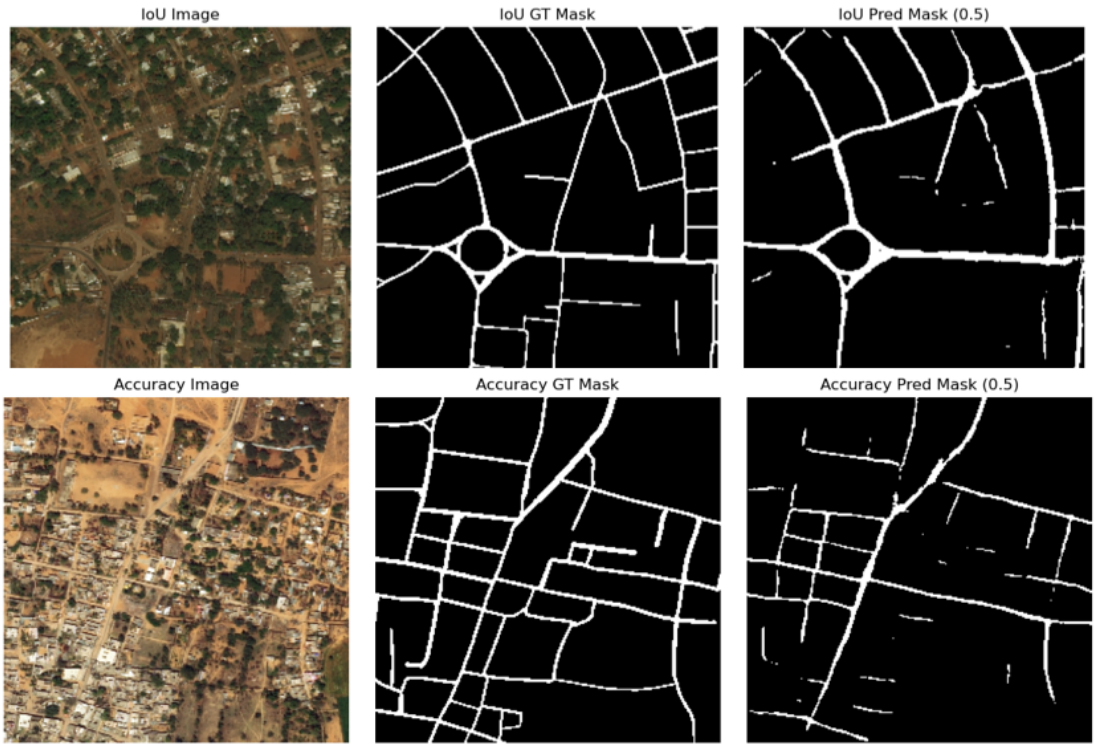

Figure 3: Comparison of IoU Model and BCE Model

The IoU-trained model makes road masks that stay much more continuous, especially in curved areas or around roundabouts. IT also keeps thin roads that the accuracy-trained model often breaks or misses. The accuracy model tends to leave gaps in the image because accuracy is dominated by background pixels, so it gets "rewarded" even when it under-predicts roads. As seen in Figure 3, the IoU model follows the actual street layout more closely, while the accuracy model looks more patchy or incomplete. This shows why accuracy alone isn't a good measure for this kind of systematic segregation and why IoU training fits the problem better.

4 Experience

Being apart of this Machine Learning CI was honestly one of my favorite parts of the semester, especially as a freshman. This project made me feel like I was getting the most out of my education because I was actually working on a project that I enjoyed. I also really enjoyed the classes and notebooks that we worked on at the beginning of the semester to prepare us for this project.

One thing that definitely affected my project was the GPU issues I had with Palmetto. For the first half of the semester, my GPU was working fine, which made every epoch only take about 2.5 hours. For some reason, during the second half of the semester, whenever I would run my program on GPU, it would produce an error leaving me no choice but to run it on CPU which took 8.5 hours per epoch. This definitely slowed me down, but for the next semester, I will look deeper into this issue and try to contact palmetto about it.

This CI also helped me pick up a lot of skills that i would usually not get in my undergraduate classes, especially as a freshman. I learned so much about machine learning, building a U-Net, how to prepare large datasets, and how to evaluate segmentation results both numerically and visually. Overall, I really enjoyed this CI. Even with the challenges, it made me more excited about AI and computer vision, and I'm definitely ready to build on this next semester.

5 Conclusions and Future Work

Working on this project showed me just how much of a difference the right metric can make. Even both the models almost reached the same accuracy, the IoU-trained model did a better job of actually finding the roads. For next semester, I technically have the option to switch to a totally new project, but I'd rather keep going with something related to this. I'm rally interested in taking it beyond this basic road model, maybe going into something closer to a self-driving car vision system. Instead of prediction just one class, I would love to do multiple things at once, like roads, people cars, bikes, traffic lights, or whatever else is in the scene. I would have to use a different type of model, like Mask R-CNN, YOLO or DeepLab, and working with more complex datasets. I'm not specifically sure what exactly what to do yet, but this CI has definitely made me want to do more complex computer vision projects.